

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the right of the index's right.

0	1	2	3	4	5
1	7	3	6	5	6

output = 3

The pivot index = 3

left sum = 11

right sum = 11

0	1	2
2	1	-1

0 ↑ 0

pivot index = 0

optimized

using prefix sum

original Array

1	7	3	6	5	6
0	1	2	3	4	5

prefix

1	8	11	17	22	28
0	1	2	3	4	5

Suffix

28	27	20	17	11	6
0	1	2	3	4	5

using prefix and suffix sum array

TC got reduced to $O(n)$ and SC $O(n)$

Brute Force

$O(n^2)$

The most basic idea is to use nested loops. The outer loop iterates through all the indices one by one. Consider it as equilibrium index. The inner loop finds out whether index i is equilibrium index or not, by checking if left side sum = right side sum.

Better Way

TC: $O(n)$

SC: $O(n)$

The idea is to remove the need of inner loop. Instead of calculating the left side sum and right side sum each time, precompute the prefix sum array and suffix sum array, and simply access this in $O(1)$ time.

So for each index i , we can check if $\text{prefixSum}[i - 1] = \text{suffixSum}[i + 1]$ but to avoid unnecessary boundary checks we can also check if $\text{prefixSum}[i] = \text{suffixSum}[i]$.

```
#include<bits/stdc++.h>
using namespace std;
```

```
/* int findEquilibriumIndex(vector<int>& arr){ // TC ->  $O(n^2)$ 
    for(int i=0;i<arr.size();i++){
        int leftSum=0,rightSum=0;
        for(int j=0;j<i;j++){
            leftSum+=arr[j];
        }
        for(int j=i+1;j<arr.size();j++){
            rightSum+=arr[j];
        }
        if(leftSum==rightSum){
            return i;
        }
    }
    return -1;
} */
```

Brute Force

```
/* int findEquilibriumIndex(vector<int>& arr){ // TC :  $O(N)$  & SC :  $O(N)$ 
    int n=arr.size();

    vector<int> pref(n,0);
    vector<int> suff(n,0);

    pref[0]=arr[0];
    suff[n-1]=arr[n-1];

    for(int i=1;i<n;i++){
        pref[i]=pref[i-1]+arr[i];
    }
    for(int i=n-2;i>=0;i--){
        suff[i]=suff[i+1]+arr[i];
    }

    for(int i=0;i<n;i++){
        if(pref[i]==suff[i]) return i;
    }
    return -1;
} */
```

Method

```
int findEquilibriumIndex(vector<int> arr) { // TC :  $O(N)$  & SC :  $O(1)$ 
    int n=arr.size(),total=0;
    for(int i=0;i<n;i++){
        total+=arr[i];
    }
    int sum=0;
    for(int i=0;i<n;i++){
        if(sum==total-sum-arr[i]){
            return (i);
        }else{
            sum+=arr[i];
        }
    }
    return -1;
}
```

Best

$O(N)$
$O(1)$

```
int main(){
    vector<int> arr={1,7,3,6,5,6};
    // vector<int> arr={1,1,1,1};
    cout<<"The pivot index is : "<<findEquilibriumIndex(arr)<<endl;
    return 0;
}
```